

CSCI 5380

Network Virtualization and Orchestration

Lab 10

Kubernetes Networking

University of Colorado Boulder
Department of Computer Science
Network Engineering

Professor Levi Perigo, Ph.D.

Summary

With modern web services, users expect applications to be available 24/7, and developers expect to deploy new versions of those applications several times a day. Containerization helps package software to serve these goals, enabling applications to be released and updated in an easy and fast way without downtime. Kubernetes helps you make sure those containerized applications run where and when you want and helps them find the resources and tools they need to work. Kubernetes is a production-ready, open-source platform designed with Google's accumulated experience in container orchestration, combined with best-of-breed ideas from the community.

A Kubernetes cluster consists of two types of resources:

- **Master node:** The master coordinates all activities in your cluster, such as scheduling applications, maintaining applications' desired state, scaling applications, and rolling out new updates.
- **Worker node:** Each node has a Kubelet, which is an agent for managing the node and communicating with the Kubernetes master. The node should also have tools for handling container operations, such as Docker or rkt.

When you deploy applications on Kubernetes, you tell the master to start the application containers. The master schedules the containers to run on the cluster's nodes. The nodes communicate with the master using the Kubernetes API, which the master exposes. A Kubernetes *Pod* is a group of one or more Containers, tied together for the purposes of administration and networking. End users can also use the Kubernetes API directly to interact with the cluster.

To get started with Kubernetes development, you can use *Minikube* or *Kubeadm*. Minikube is a lightweight all-in-one Kubernetes implementation that creates a VM on your local machine and deploys a simple cluster containing only one node. The Minikube CLI provides basic operations for working with your cluster, including start, stop, status, and delete. Kubeadm is used for a multi-node setup where one node acts as Master Node and other nodes act as Worker nodes. Kubeadm tool helps you bootstrap your Kubernetes cluster.

Objective 1 – Environment Setup (VM and Kubeadm)

This objective focuses on setting up a multimode Kubernetes Cluster. You are required to follow the requirements mentioned in the “Before you Begin Section” stated [here](#). As you follow the below steps, please be sure to check the updated requirements in Kubernetes documentation available [here](#).

VM Setup:

We will be using 3 VMs as three different Kubernetes cluster nodes described in Fig. 1. You can use your laptop provided you have 8 GB of RAM (2GB for each VM) and 6-8 CPU cores (2 cores for each VM). Alternatively, you may use your Lab server having relatively new Ubuntu Desktop/Minimal versions (Ubuntu 22.04 is recommended) with Virtualbox as your provider for setting up VMs.

Each VM should have bridged (or NAT) and host-only adapters for Internet and Internal connectivity. Host-only interface should be statically configured (192.168.56.X/24) and should have unique hostnames – “kubemaster” for kubernetes “master: VM 1”, “kubenode1” for kubernetes “worker1: VM 2” and “kubenode2” for kubernetes “worker2: VM 3”. Add entries in the file /etc/hosts on each VM, these new entries on should be the same on all VMs that highlight the host-only IP address and the hostname of all VMs. Additionally, please make sure SSH service is active across all VMs. Disable Swap across all VMs using “swapoff -a” command and comment the *swap* line (typically last line in the file) in /etc/fstab (do not modify any lines in this file as it contains boot information).

- Paste the screenshots showing the content of the file /etc/hosts on each VM after adding new entries. **[10 points]**

I deployed the VM's in openstack –

☐	Instance Name ▲	Image Name	IP Address	Flavor	Key Pair	Status	Availability Zone
☐	kubemaster	debian	192.168.50.113, 172.24.4.191	m1.small	-	Active	nova
☐	kubenode1	debian	192.168.50.154, 172.24.4.166	m1.small	-	Active	nova
☐	kubenode2	debian	192.168.50.16, 172.24.4.243	m1.small	-	Active	nova

I have added the hosts entry on VM for all three nodes, and they are reachable as well. I had to configure ICMP and SSH to the instances to ensure connectivity is there.

- On each VM, ping the hostname of the other two VMs, and paste the screenshots indicating the ping results. **[10 points]**

VM HOST-

```
atul@csci5380-vm2-atan8167:~$ sudo cat /etc/hosts | grep kube
172.24.4.191 kubemaster
172.24.4.166 kubenode1
172.24.4.243 kubenode2
atul@csci5380-vm2-atan8167:~$ ping kubenode1
PING kubenode1 (172.24.4.166) 56(84) bytes of data.
64 bytes from kubenode1 (172.24.4.166): icmp_seq=1 ttl=63 time=1.72 ms
64 bytes from kubenode1 (172.24.4.166): icmp_seq=2 ttl=63 time=0.639 ms
^C
--- kubenode1 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 0.639/1.177/1.715/0.538 ms
atul@csci5380-vm2-atan8167:~$ ping kubenode2
PING kubenode2 (172.24.4.243) 56(84) bytes of data.
64 bytes from kubenode2 (172.24.4.243): icmp_seq=1 ttl=63 time=1.84 ms
^C
--- kubenode2 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 1.841/1.841/1.841/0.000 ms
atul@csci5380-vm2-atan8167:~$ ping kubemaster
PING kubemaster (172.24.4.191) 56(84) bytes of data.
64 bytes from kubemaster (172.24.4.191): icmp_seq=1 ttl=63 time=1.86 ms
64 bytes from kubemaster (172.24.4.191): icmp_seq=2 ttl=63 time=0.855 ms
^C
--- kubemaster ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 0.855/1.359/1.864/0.504 ms
```

kubemaster-

```
debian@kubemaster:~$ sudo cat /etc/hosts | grep kube
127.0.1.1 kubemaster.novalocal kubemaster
172.24.4.243 kubenode1
172.24.4.191 kubenode2
debian@kubemaster:~$ ping kubenode1
PING kubenode1 (172.24.4.243) 56(84) bytes of data.
64 bytes from kubenode1 (172.24.4.243): icmp_seq=1 ttl=62 time=2.21 ms
64 bytes from kubenode1 (172.24.4.243): icmp_seq=2 ttl=62 time=0.811 ms
^C
--- kubenode1 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 3ms
rtt min/avg/max/mdev = 0.811/1.508/2.206/0.698 ms
debian@kubemaster:~$ ping kubenode2
PING kubenode2 (172.24.4.191) 56(84) bytes of data.
64 bytes from kubenode2 (172.24.4.191): icmp_seq=1 ttl=62 time=1.66 ms
64 bytes from kubenode2 (172.24.4.191): icmp_seq=2 ttl=62 time=0.937 ms
^C
--- kubenode2 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 3ms
rtt min/avg/max/mdev = 0.937/1.296/1.656/0.361 ms
```

kubenode1-

```
debian@kubenode1:~$ sudo cat /etc/hosts | grep kube
127.0.1.1 kubenode1.novalocal kubenode1
172.24.4.191 kubenode2
172.24.4.166 kubemaster
debian@kubenode1:~$ ping kubenode2
PING kubenode2 (172.24.4.191) 56(84) bytes of data.
64 bytes from kubenode2 (172.24.4.191): icmp_seq=1 ttl=62 time=1.97 ms
64 bytes from kubenode2 (172.24.4.191): icmp_seq=2 ttl=62 time=1.41 ms
^C
--- kubenode2 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 2ms
rtt min/avg/max/mdev = 1.413/1.690/1.968/0.280 ms
debian@kubenode1:~$ ping kubemaster
PING kubemaster (172.24.4.166) 56(84) bytes of data.
64 bytes from kubemaster (172.24.4.166): icmp_seq=1 ttl=62 time=1.91 ms
64 bytes from kubemaster (172.24.4.166): icmp_seq=2 ttl=62 time=1.47 ms
^C
--- kubemaster ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 2ms
rtt min/avg/max/mdev = 1.470/1.692/1.914/0.222 ms
```

kubenode2-

```
debian@kubenode2:~$ cat /etc/hosts | grep kube
127.0.1.1 kubenode2.novalocal kubenode2
172.24.4.166 kubemaster
172.24.4.243 kubenode1
debian@kubenode2:~$ ping kubemaster
PING kubemaster (172.24.4.166) 56(84) bytes of data.
64 bytes from kubemaster (172.24.4.166): icmp_seq=1 ttl=62 time=1.90 ms
64 bytes from kubemaster (172.24.4.166): icmp_seq=2 ttl=62 time=1.39 ms
^C
--- kubemaster ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 2ms
rtt min/avg/max/mdev = 1.385/1.641/1.897/0.256 ms
debian@kubenode2:~$ ping kubenode1
PING kubenode1 (172.24.4.243) 56(84) bytes of data.
64 bytes from kubenode1 (172.24.4.243): icmp_seq=1 ttl=62 time=1.68 ms
64 bytes from kubenode1 (172.24.4.243): icmp_seq=2 ttl=62 time=1.25 ms
^C
--- kubenode1 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 3ms
rtt min/avg/max/mdev = 1.249/1.463/1.677/0.214 ms
debian@kubenode2:~$
```

Kubeadm Setup:

Refer to Fig. 1 for high level steps regarding kubeadm installation steps on each VM.

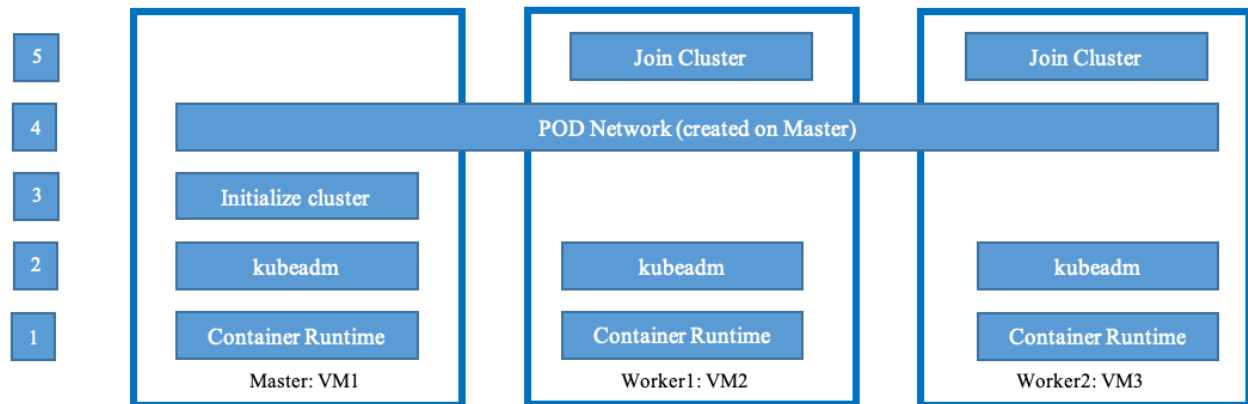


Fig. 1: Installation Steps

Step 1:

Before proceeding with Kubeadm setup, install a [container runtime](#) for example containerd (recommended) or Docker across all the VMs.

Enabled the ip forwarding on the nodes –

We had to install using wget since the Debian version was old and it caused issue with repo –

Following commands -

```
wget https://github.com/containerd/containerd/releases/download/v1.6.12/containerd-1.6.12-linux-amd64.tar.gz
sudo tar Cxzf /usr/local containerd-1.6.12-linux-amd64.tar.gz
/usr/local/bin/containerd --version
sudo mkdir -p /etc/containerd
containerd config default | sudo tee /etc/containerd/config.toml
```

Then I edited the value for the configured SystemdCgroup = true in config file under etc folder.

Kubemaster -

```
debian@kubemaster:~$ sudo sysctl -w net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1
debian@kubemaster:~$ █
containerd-1.6.12-linux-amd64.tar.gz 100%[=====]
2026-04-06 21:07:11 (63.5 MB/s) - 'containerd-1.6.12-linux-amd64.tar.gz' saved [43433790/43433790]

debian@kubemaster:~$
debian@kubemaster:~$ sudo tar Cxzf /usr/local containerd-1.6.12-linux-amd64.tar.gz
bin/
bin/ctr
bin/containerd
bin/containerd-shim
bin/containerd-stress
bin/containerd-shim-runc-v2
bin/containerd-shim-runc-v1
debian@kubemaster:~$ /usr/local/bin/containerd --version
containerd github.com/containerd/containerd v1.6.12 a05d175400b1145e5e6a735a6710579d181e7fb0
debian@kubemaster:/etc/containerd$ █
debian@kubemaster:/etc/containerd$ sudo systemctl daemon-reload
debian@kubemaster:/etc/containerd$ sudo systemctl enable --now containerd
Created symlink /etc/systemd/system/multi-user.target.wants/containerd.service → /etc/systemd/system/containerd.service.
debian@kubemaster:/etc/containerd$ sudo systemctl status containerd
● containerd.service - containerd container runtime
   Loaded: loaded (/etc/systemd/system/containerd.service; enabled; vendor preset: enabled)
   Active: active (running) since Mon 2026-04-06 21:13:40 UTC; 5s ago
     Docs: https://containerd.io
  Process: 1028 ExecStartPre=sbin/modprobe overlay (code=exited, status=0/SUCCESS)
    Main PID: 1030 (containerd)
      Tasks: 8
     Memory: 18.8M
    CGroup: /system.slice/containerd.service
            └─1030 /usr/local/bin/containerd

Apr 06 21:13:40 kubemaster containerd[1030]: time="2026-04-06T21:13:40.276719443Z" level=info msg="Start subscribing containerd event"
Apr 06 21:13:40 kubemaster containerd[1030]: time="2026-04-06T21:13:40.277031521Z" level=info msg="Start recovering state"
Apr 06 21:13:40 kubemaster containerd[1030]: time="2026-04-06T21:13:40.277405051Z" level=info msg="Start event monitor"
Apr 06 21:13:40 kubemaster containerd[1030]: time="2026-04-06T21:13:40.277651403Z" level=info msg="Start snapshots syncer"
Apr 06 21:13:40 kubemaster containerd[1030]: time="2026-04-06T21:13:40.277810838Z" level=info msg="Start cni network conf syncer for default"
Apr 06 21:13:40 kubemaster containerd[1030]: time="2026-04-06T21:13:40.277948108Z" level=info msg="Start streaming server"
Apr 06 21:13:40 kubemaster containerd[1030]: time="2026-04-06T21:13:40.278799415Z" level=info msg=serving... address=/run/containerd/containerd.sock.ttrpc
Apr 06 21:13:40 kubemaster containerd[1030]: time="2026-04-06T21:13:40.279005000Z" level=info msg=serving... address=/run/containerd/containerd.sock
Apr 06 21:13:40 kubemaster systemd[1]: Started containerd container runtime.
Apr 06 21:13:40 kubemaster containerd[1030]: time="2026-04-06T21:13:40.284287485Z" level=info msg="containerd successfully booted in 0.059939s"
```

Worker2-

```
debian@kubenode2:~$ sysctl net.ipv4.ip_forward
net.ipv4.ip_forward = 0
debian@kubenode2:~$ sudo sysctl -w net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1
debian@kubenode2:~$ █
debian@kubenode2:~$ sudo systemctl daemon-reload
debian@kubenode2:~$ sudo systemctl enable --now containerd
Created symlink /etc/systemd/system/multi-user.target.wants/containerd.service → /etc/systemd/system/containerd.service.

debian@kubenode2:~$ sudo systemctl status containerd
● containerd.service - containerd container runtime
   Loaded: loaded (/etc/systemd/system/containerd.service; enabled; vendor preset: enabled)
   Active: active (running) since Mon 2026-04-06 21:17:34 UTC; 1min 1s ago
     Docs: https://containerd.io
  Process: 1028 ExecStartPre=sbin/modprobe overlay (code=exited, status=0/SUCCESS)
    Main PID: 1030 (containerd)
      Tasks: 8
     Memory: 18.8M
    CGroup: /system.slice/containerd.service
            └─1030 /usr/local/bin/containerd
```

Worker1-

```
debian@kubenode1:~$ sysctl net.ipv4.ip_forward
net.ipv4.ip_forward = 0
debian@kubenode1:~$ sudo sysctl -w net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1
debian@kubenode1:~$
```

```
debian@kubenode1:~$ sudo systemctl daemon-reload
debian@kubenode1:~$ sudo systemctl enable --now containerd
Created symlink /etc/systemd/system/multi-user.target.wants/containerd.service.

debian@kubenode1:~$ sudo systemctl status containerd
● containerd.service - containerd container runtime
   Loaded: loaded (/etc/systemd/system/containerd.service; enabled; vendor preset: enabled)
   Active: active (running) since Mon 2026-04-06 21:16:00 UTC; 22ms ago
     Docs: https://containerd.io
```

Step 2:

There are plenty of resources online that can help you install Kubeadm across all VMs or you may follow below commands:

```
sudo apt-get update
sudo apt install -y curl gnupg2 software-properties-common apt-transport-https ca-certificates
curl -s https://packages.cloud.google.com/apt/doc/apt-key.gpg | sudo apt-key add -
cat <<EOF | sudo tee /etc/apt/sources.list.d/kubernetes.list
deb https://apt.kubernetes.io/ kubernetes-xenial main
EOF
sudo apt-get update
sudo apt-get install -y kubelet kubeadm kubectl
sudo apt-mark hold kubelet kubeadm kubectl
```

For more information, you may follow this [link](#).

I installed the kubectl on all the nodes –

I had to run ignore preflight checks since Debian version was old –

```
sudo kubeadm init --pod-network-cidr=172.24.0.0/16 --apiserver-advertise-address=172.24.4.166 --ignore-preflight-errors=NumCPU
```

Since I had deployed a flavour which had only 1 CPU I had included to ignore this precheck

Step 3: Initialize Kubernetes on the Master node (VM 1)

Run the following command to initialize Kubernetes on the Master node:

```
kubeadm init --pod-network-cidr=192.168.0.0/16 --apiserver-advertise-address=<Master Node's Host-only Static IP>
```

- Paste the screenshot showing the output of the above command. [10 points]

```
[addons] Applied essential addon: kube-proxy

Your Kubernetes control-plane has initialized successfully!

To start using your cluster, you need to run the following as a regular user:

  mkdir -p $HOME/.kube
  sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
  sudo chown $(id -u):$(id -g) $HOME/.kube/config

Alternatively, if you are the root user, you can run:

  export KUBECONFIG=/etc/kubernetes/admin.conf

You should now deploy a pod network to the cluster.
Run "kubectl apply -f [podnetwork].yaml" with one of the options listed at:
  https://kubernetes.io/docs/concepts/cluster-administration/addons/

Then you can join any number of worker nodes by running the following on each as root:

kubeadm join 192.168.50.45:6443 --token w3rkke.1u15af980lxtnl1i2 \
--discovery-token-ca-cert-hash sha256:4520f8234fdaedc6c7416c50ed1604541227b281706488a657c10df71ad3b0f8
debian@kubemaster:~$
```

Ran the following command as per message in screenshot –

```
mkdir -p $HOME/.kube
```

```
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
```

```
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Once the “kubeadm init” command completes its execution, run the required commands shown in the output to start using the cluster. The output will show a command in its output (toward the end) that starts with “kubeadm join --token”. Copy and save that command since we will be using it to join worker nodes to the Master node.

```
debian@kubemaster:~$ mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
debian@kubemaster:~$ kubectl cluster-info
```

- Run the command “kubectl cluster-info”, and paste the screenshot of its output. [5 points]

```
debian@kubemaster:~$ kubectl cluster-info
Kubernetes control plane is running at https://192.168.50.45:6443
CoreDNS is running at https://192.168.50.45:6443/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
debian@kubemaster:~$
```

Step 4: Install POD Network

In this step you'll setup a POD network so that Kubernetes *pods* can communicate with each other. You may use Calico or Flannel to setup this internal network. In this lab, we'll be using Calico. On your Master node (VM 1) run following commands to setup [Calico](#):

```
kubectl apply -f
https://docs.projectcalico.org/v3.11/manifests/calico.yaml
```

Once a pod network has been installed, you can confirm that it is working by running the command “`kubectl get pods --all-namespaces`”. Paste the screenshot showing the output of this command. [5 points]

```
debian@kubemaster:~$ kubectl apply -f https://raw.githubusercontent.com/projectcalico/calico/v3.26.1/manifests/
poddisruptionbudget.policy/calico-kube-controllers created
serviceaccount/calico-kube-controllers created
serviceaccount/calico-node created
serviceaccount/calico-cni-plugin created
configmap/calico-config created
customresourcedefinition.apiextensions.k8s.io/bgpconfigurations.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/bgpfilters.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/bgppeers.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/blockaffinities.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/caliconodestatuses.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/clusterinformations.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/felixconfigurations.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/globalnetworkpolicies.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/globalnetworksets.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/hostendpoints.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/ipamblocks.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/ipamconfigs.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/ipamhandles.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/ippools.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/ippreservations.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/kubecontrollersconfigurations.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/networkpolicies.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/networksets.crd.projectcalico.org created
clusterrole.rbac.authorization.k8s.io/calico-kube-controllers created
clusterrole.rbac.authorization.k8s.io/calico-node created
clusterrole.rbac.authorization.k8s.io/calico-cni-plugin created
clusterrolebinding.rbac.authorization.k8s.io/calico-kube-controllers created
clusterrolebinding.rbac.authorization.k8s.io/calico-node created
clusterrolebinding.rbac.authorization.k8s.io/calico-cni-plugin created
daemonset.apps/calico-node created
deployment.apps/calico-kube-controllers created
```

```
debian@kubemaster:~$ kubectl get pods --all-namespaces
NAMESPACE   NAME                                     READY   STATUS             RESTARTS   AGE
kube-system  calico-kube-controllers-689744956f-qmhb2 0/1     ContainerCreating  0          32s
kube-system  calico-node-nff7g                         0/1     Running            0          32s
kube-system  coredns-668d6bf9bc-592pm                 0/1     ContainerCreating  0          4m42s
kube-system  coredns-668d6bf9bc-rs74w                 0/1     Pending           0          4m42s
kube-system  etcd-kubemaster                          1/1     Running            0          4m47s
kube-system  kube-apiserver-kubemaster                 1/1     Running            0          4m47s
kube-system  kube-controller-manager-kubemaster        1/1     Running            0          4m50s
kube-system  kube-proxy-bgzkl                          1/1     Running            0          4m42s
kube-system  kube-scheduler-kubemaster                 1/1     Running            0          4m47s
```

Step 5: Join Worker nodes (VM 2 and VM 3) to Master node (VM 1)

Here pastes the command (on VM 2 and VM 3) that you copied in Step 3 from the output of “kubeadm init” command. Once the commands are done executing, run “kubectl get nodes” command on Master node (VM 1) which should show VM 2 and VM 3 in the kubernetes cluster and their status is “Ready”.

- Paste the screenshot showing the output of running the command “kubectl get nodes” on Master node. [10 points]

```
debian@kubenode1:~$ sudo kubeadm join 192.168.50.45:6443 --token w3rkke.1u15af980lxtn1i2 \
--discovery-token-ca-cert-hash sha256:4520f8234fdaedc6c7416c50ed1604541227b281706488a657c10df71ad3b0f8 \
--ignore-preflight-errors=NumCPU
[preflight] Running pre-flight checks
[preflight] Reading configuration from the "kubeadm-config" ConfigMap in namespace "kube-system"...
[preflight] Use 'kubeadm init phase upload-config --config your-config.yaml' to re-upload it.
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/config.yaml"
[kubelet-start] Writing kubelet environment file with flags to file "/var/lib/kubelet/kubeadm-flags.env"
[kubelet-start] Starting the kubelet
[kubelet-check] Waiting for a healthy kubelet at http://127.0.0.1:10248/healthz. This can take up to 4m0s
[kubelet-check] The kubelet is healthy after 517.846795ms
[kubelet-start] Waiting for the kubelet to perform the TLS Bootstrap

This node has joined the cluster:
* Certificate signing request was sent to apiserer and a response was received.
* The Kubelet was informed of the new secure connection details.

Run 'kubectl get nodes' on the control-plane to see this node join the cluster.

debian@kubenode1:~$
```

```
debian@kubenode2:~$ sudo kubeadm join 192.168.50.45:6443 --token w3rkke.1u15af980lxtn1i2 \
--discovery-token-ca-cert-hash sha256:4520f8234fdaedc6c7416c50ed1604541227b281706488a657c10df71ad3b0f8 \
--ignore-preflight-errors=NumCPU
[preflight] Running pre-flight checks
[preflight] Reading configuration from the "kubeadm-config" ConfigMap in namespace "kube-system"...
[preflight] Use 'kubeadm init phase upload-config --config your-config.yaml' to re-upload it.
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/config.yaml"
[kubelet-start] Writing kubelet environment file with flags to file "/var/lib/kubelet/kubeadm-flags.env"
[kubelet-start] Starting the kubelet
[kubelet-check] Waiting for a healthy kubelet at http://127.0.0.1:10248/healthz. This can take up to 4m0s
[kubelet-check] The kubelet is healthy after 1.003157858s
[kubelet-start] Waiting for the kubelet to perform the TLS Bootstrap

This node has joined the cluster:
* Certificate signing request was sent to apiserer and a response was received.
* The Kubelet was informed of the new secure connection details.

Run 'kubectl get nodes' on the control-plane to see this node join the cluster.

debian@kubenode2:~$
```

Master Node –

```
debian@kubemaster:~$ kubectl get nodes
NAME             STATUS    ROLES    AGE     VERSION
kubemaster       Ready    control-plane   8m22s   v1.32.13
kubenode1        Ready    <none>        91s     v1.32.13
kubenode2        Ready    <none>        59s     v1.32.13
debian@kubemaster:~$
```

Objective 2

In this objective, you'll create a multi-node Kubernetes cluster of SDN controllers to achieve high-availability (HA) using Kubeadm as shown in the below Fig. 2 and scale the *Pods*. You may choose any SDN controller of your choice.

Kubernetes reference: <https://kubernetes.io/docs/concepts/>

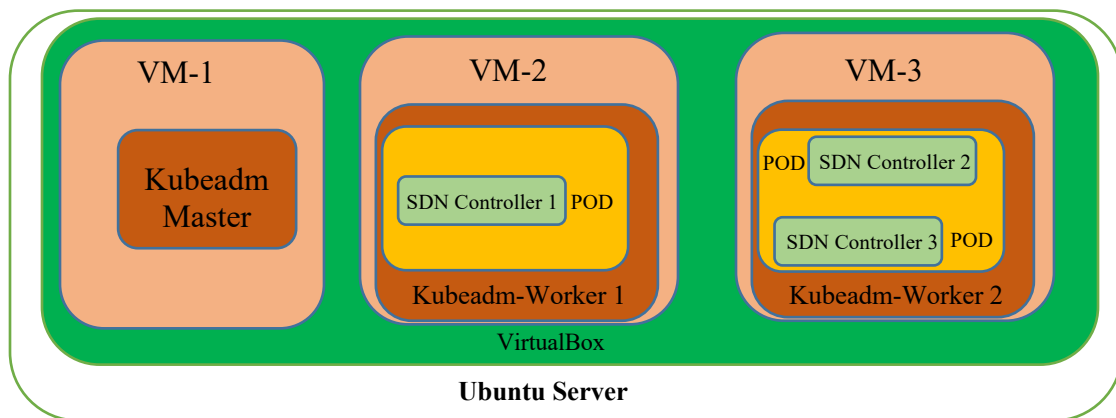


Fig. 2: Multi-node Kubernetes cluster of SDN controllers

1. Paste the screenshot indicating number of nodes in the cluster. **[5 points]**

```
debian@kubemaster:~$ kubectl get nodes
NAME             STATUS    ROLES    AGE   VERSION
kubemaster       Ready    control-plane   8m22s   v1.32.13
kubenode1        Ready    <none>         91s    v1.32.13
kubenode2        Ready    <none>         59s    v1.32.13
debian@kubemaster:~$
```

2. Create a Pod consisting of single SDN Controller by writing a YAML definition file of Pod/deployment. Paste the screenshot showing the output of some command indicating the created Pod. **[5 points]**

```
debian@kubemaster:~$ kubectl apply -f ryu.yaml
pod/sdn-controller created
debian@kubemaster:~$ kubectl get pods -w
NAME             READY    STATUS    RESTARTS   AGE
sdn-controller   1/1      Running   0           3s
^Cdebian@kubemaster:~$
```

3. Create a ReplicaSet for a cluster of 3 Pods of SDN controllers (one in each Pod) by writing a ReplicaSet definition (YAML) file. Paste the screenshots of some commands indicating the 3 replicas of SDN controller, and the worker node on which each Pod was deployed? **[5 points]**

```
q^Cdebian@kubemaster:~$ kubectl get pods -w
NAME                                READY   STATUS    RESTARTS   AGE
sdn-replicaset-7mxzm                1/1     Running   0           21s
sdn-replicaset-f74dd                1/1     Running   0           21s
sdn-replicaset-k4cfx                1/1     Running   0           21s
```

I have attached the Yaml file in the submission.

4. Create a NodePort service by writing a NodePort definition file to expose controller's IP/Port to make an OpenFlow connection from a Mininet host outside the Kubernetes virtual network (You can run a Mininet agent on your laptop or using the Mininet VM from NetMan). Paste the screenshot of the output of command "kubectl get services" to indicate the created NodePort service. **[10 points]**

```
^Cdebian@kubemaster:~$ kubectl get services
NAME            TYPE        CLUSTER-IP    EXTERNAL-IP  PORT(S)          AGE
kubernetes      ClusterIP   10.96.0.1     <none>       443/TCP          27m
sdn-service     NodePort    10.104.66.245 <none>       6633:30644/TCP  16s
```

5. Create a Mininet network and connect the switches to the sdn controller application running on the 3 pods in the cluster. Paste the screenshot of your command to create the mininet network and the result of the command "pingall" in the mininet network. **[10 points]**

```
h1 h2 h3 h4
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1) (h3, s1) (h4, s1)
*** Configuring hosts
h1 h2 h3 h4
*** Starting controller

*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 h3
*** Results: 0% dropped (12/12 received)
```

6. Indicate that controller failover works (by shutting down one of the Pods). Did Kubernetes maintain the ReplicaSet number defined in the definition file? Support your answer by pasting the related screenshots. [5 points]

We can see in screenshot that as soon as one node was terminated it took over another instance

```
debian@kubemaster:~$ kubectl get pods -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP            NODE       NOMINATED NODE   READINESS GATES
sdn-replicaset-7qxp  1/1     Running   1 (87s ago)  5m32s  192.168.35.66  kubemaster2  <none>           <none>
sdn-replicaset-rw96q  1/1     Running   1 (86s ago)  5m32s  192.168.205.195  kubemaster1  <none>           <none>
sdn-replicaset-sjprv  1/1     Running   1 (86s ago)  5m32s  192.168.205.196  kubemaster1  <none>           <none>
debian@kubemaster:~$ kubectl delete pod sdn-replicaset-7qxp
pod "sdn-replicaset-7qxp" deleted from default namespace
^Cdebian@kubemaster:~$ kubectl get pods -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP            NODE       NOMINATED NODE   READINESS GATES
sdn-replicaset-6gg2r  1/1     Running   0          8s    192.168.35.67  kubemaster2  <none>           <none>
sdn-replicaset-7qxp  1/1     Terminating 1 (98s ago)  5m43s  192.168.35.66  kubemaster2  <none>           <none>
sdn-replicaset-rw96q  1/1     Running   1 (97s ago)  5m43s  192.168.205.195  kubemaster1  <none>           <none>
sdn-replicaset-sjprv  1/1     Running   1 (97s ago)  5m43s  192.168.205.196  kubemaster1  <none>           <none>
debian@kubemaster:~$ kubectl get pods -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP            NODE       NOMINATED NODE   READINESS GATES
sdn-replicaset-6gg2r  1/1     Running   0          16s    192.168.35.67  kubemaster2  <none>           <none>
sdn-replicaset-7qxp  1/1     Terminating 1 (106s ago)  5m51s  192.168.35.66  kubemaster2  <none>           <none>
sdn-replicaset-rw96q  1/1     Running   1 (105s ago)  5m51s  192.168.205.195  kubemaster1  <none>           <none>
sdn-replicaset-sjprv  1/1     Running   1 (105s ago)  5m51s  192.168.205.196  kubemaster1  <none>           <none>
```

7. Submit all the required YAML definition files in step 2, 3, 4 along with the lab. [10 points]

Submitted

Extra Credit

In this objective, you'll deploy [Prometheus](#) and [Grafana](#) on the Master node (VM1) to monitor and visualize the status of your cluster.

1. Create a Prometheus deployment and a NodePort service to expose Prometheus on some ports. Paste screenshots of the output of the commands "**kubectl get deployment <-n namespace_name (if needed)>**" and "**kubectl get service <-n namespace_name (if needed)>**" to indicate your successful creations of Prometheus. [10 points]
2. Create a Grafana deployment (set its data source to be your Prometheus server), and a NodePort service to expose Grafana on some ports. Paste screenshots of the output of the commands "**kubectl get deployment <-n namespace_name (if needed)>**" and "**kubectl get service <-n namespace_name (if needed)>**" to indicate your successful creations of Grafana. [10 points]

3. Now you should be able to access Prometheus and Grafana from your web browser, using any of your Master node IP address and the port you set for the services.

- After open your Prometheus webpage, go to “Status -> Targets”. Select “kubernetes-nodes” from the dropdown menu, it should show that all your nodes – 1 Master node and 2 Worker nodes are up, paste a screenshot of this page. **[5 points]**
- After open your Grafana webpage (admin: admin), import [this Grafana dashboard template](#), set the refresh period to 1 minute. You should then see a chart showing the “Memory usage” of all nodes – 1 Master node and 2 Worker nodes, paste a screenshot of that chart. **[10 points]**

Total Score = _____ / 100 + 35